

Probability

IS381 - Statistics and Probability with R

Jason Bryer, Ph.D.

Week 5

Probability

There are two key properties of probability models:

1. $P(A)$ = The probability of event A
2. $0 \leq P(A) \leq 1$

This semester we will examine two interpretations of probability:

- **Frequentist interpretation:** The probability of an outcome is the proportion of times the outcome would occur if we observed the random process an infinite number of times.
- **Bayesian interpretation:** A Bayesian interprets probability as a subjective degree of belief: For the same event, two separate people could have different viewpoints and so assign different probabilities. Largely popularized by revolutionary advance in computational technology and methods during the last twenty years.

Law of Large Numbers

Law of large numbers states that as more observations are collected, the proportion of occurrences with a particular outcome, $\hat{p}_n$, converges to the probability of that outcome, p .

When tossing a fair coin, if heads comes up on each of the first 10 tosses, what do you think the chance is that another head will come up on the next coin toss? 0.5, less 0.5, or greater 0.5?

When tossing a fair coin, if heads comes up on each of the first 10 tosses, what do you think the chance is that another head will come up on the next coin toss? 0.5, less 0.5, or greater 0.5?

- The probability is still 0.5, or there is still a 50% chance that another head will come up on the next toss.
- The coin is not "due" for a tail.
- The common misunderstanding of the LLN is that random processes are supposed to compensate for whatever happened in the past; this is just not true and is also called **gambler's fallacy** (or **law of averages**).

Coin Toss Demo

```
library(DATA606)  
shiny_demo('gambler')
```

Gambler's Run

Selection the number of games to play (x axis), the odds of winning, and the number of runs (i.e. number of lines).

Number of games:



Odds of winning (1:n):



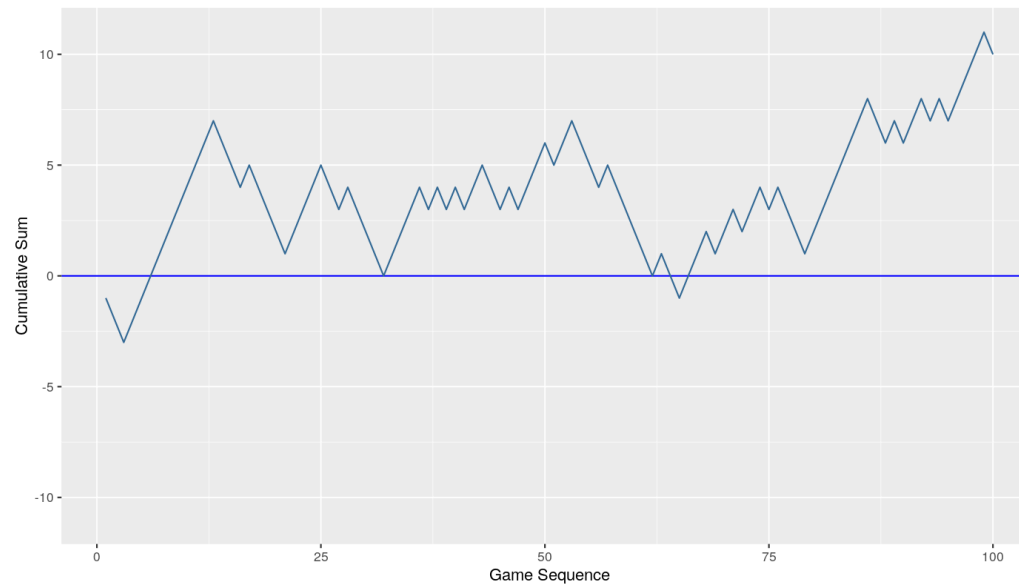
Number of runs:



Start Over

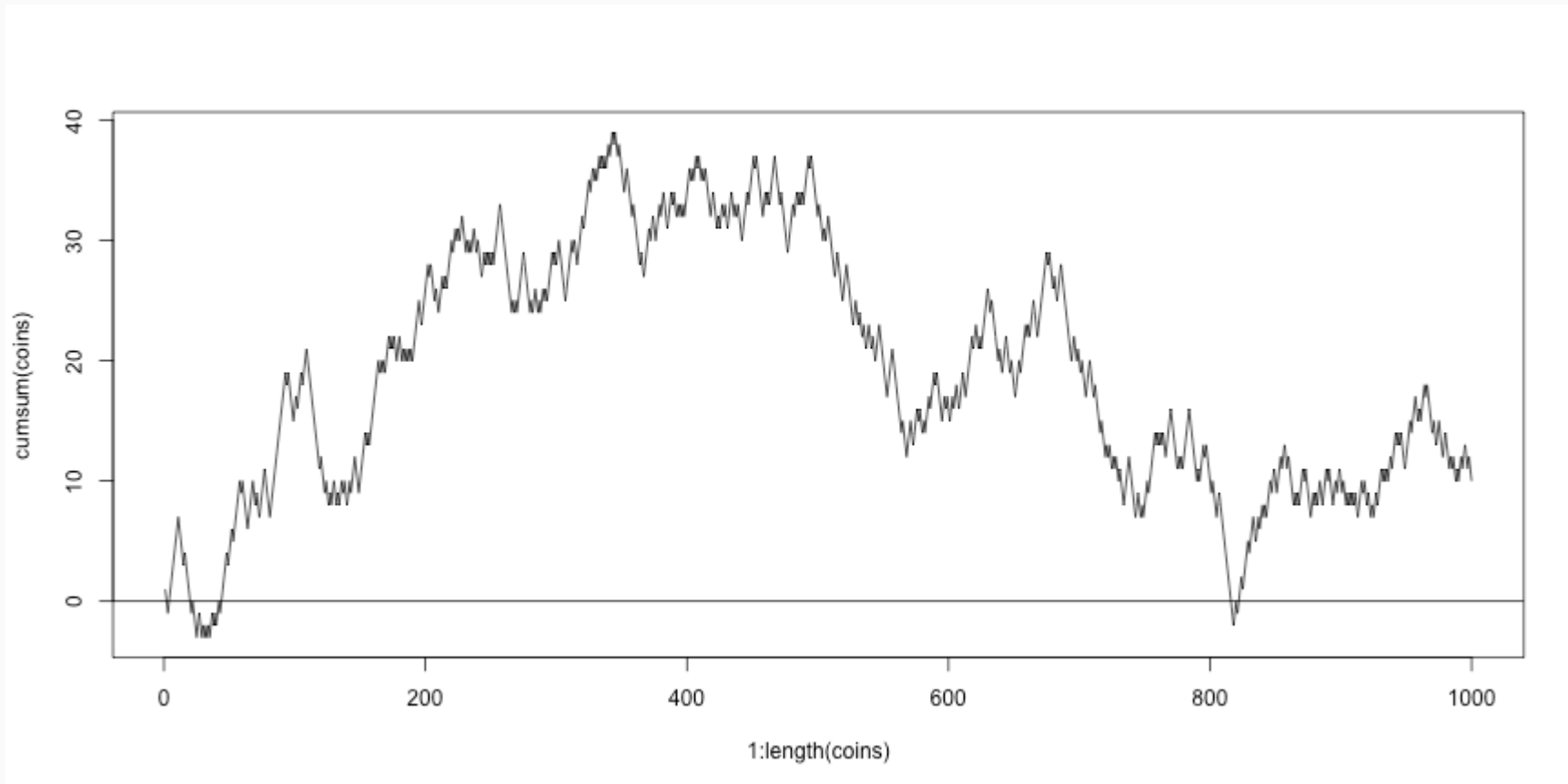
Plot [Table](#)

Average winnings after 100 games is 10



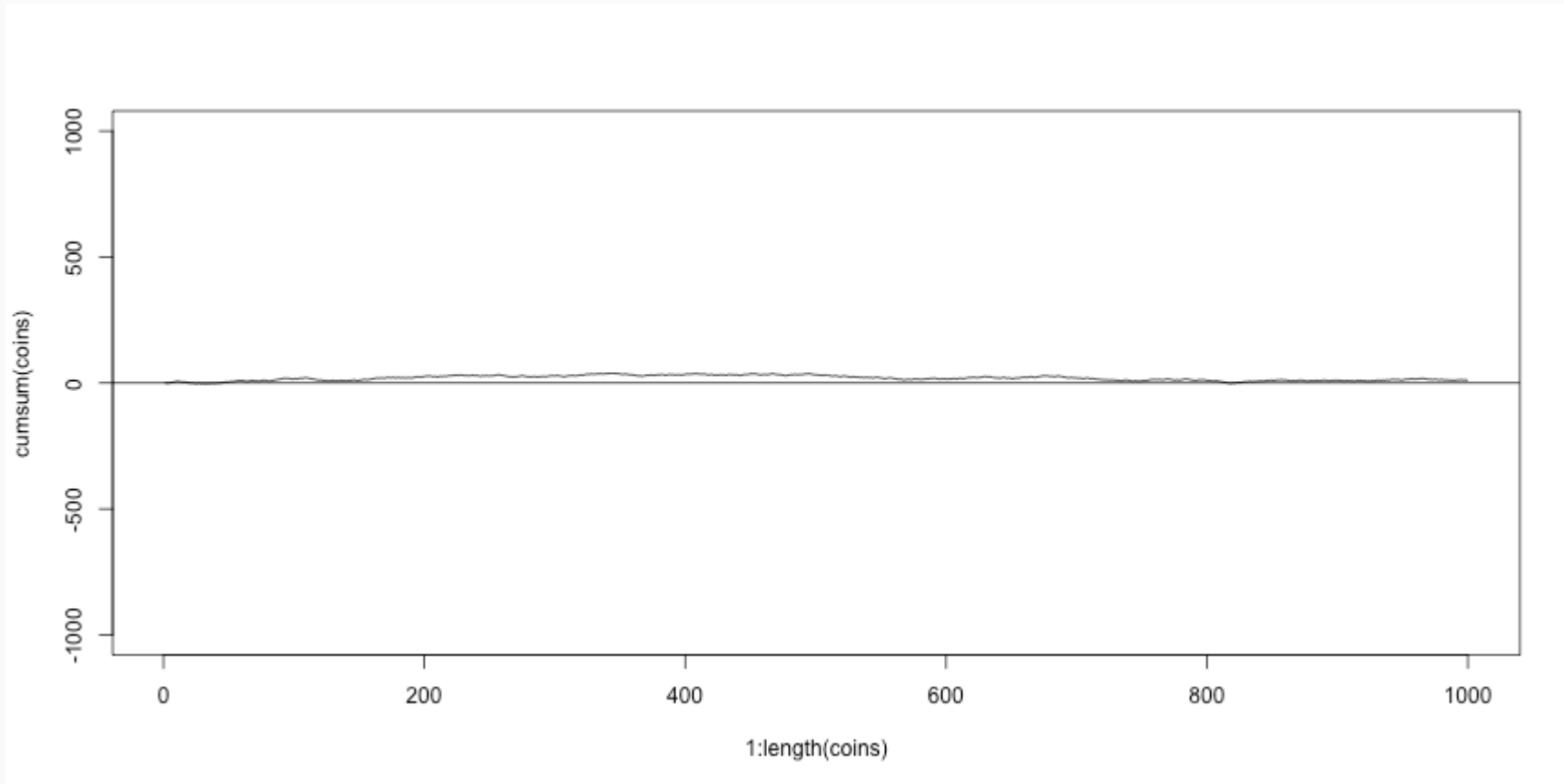
Coin Tosses

```
coins <- sample(c(-1,1), 1000, replace=TRUE)
plot(1:length(coins), cumsum(coins), type='l')
abline(h=0)
```



Coin Tosses (Full Range)

```
plot(1:length(coins), cumsum(coins), type='l', ylim=c(-1000, 1000))  
abline(h=0)
```



Disjoint and non-disjoint outcomes

Disjoint (mutually exclusive) outcomes: Cannot happen at the same time.

- The outcome of a single coin toss cannot be a head and a tail. A student both cannot fail and pass a class.
- A single card drawn from a deck cannot be an ace and a queen.

Non-disjoint outcomes: Can happen at the same time.

- A student can get an A in Stats and A in Econ in the same semester.

Probability Distributions

A probability distribution lists all possible events and the probabilities with which they occur.

- The probability distribution for the a coin toss:

Event	Heads	Tails
Probability	0.5	0.5

Rules for probability distributions:

1. The events listed must be disjoint
2. Each probability must be between 0 and 1
3. The probabilities must total 1

Probability Distributions (cont.)

The probability distribution for two coin tosses:

Event	HH	TT	HT	TH
Probability	0.25	0.25	0.25	0.25

Independence

Two processes are independent if knowing the outcome of one provides no useful information about the outcome of the other.

- Knowing that the coin landed on a head on the first toss does not provide any useful information for determining what the coin will land on in the second toss. → Outcomes of two tosses of a coin are independent.
- Knowing that the first card drawn from a deck is an ace does provide useful information for determining the probability of drawing an ace in the second draw. → Outcomes of two draws from a deck of cards (without replacement) are dependent.

Checking for Independence

If $P(A \text{ occurs, given that } B \text{ is true}) = P(A \mid B) = P(A)$, then A and B are independent.

- $P(\text{protects citizens}) = 0.58$
- $P(\text{randomly selected NC resident says gun ownership protects citizens, given that the resident is white}) = P(\text{protects citizens} \mid \text{White}) = 0.67$
- $P(\text{protects citizens} \mid \text{Black}) = 0.28$
- $P(\text{protects citizens} \mid \text{Hispanic}) = 0.64$

$P(\text{protects citizens})$ varies by race/ethnicity, therefore opinion on gun ownership and race ethnicity are most likely dependent.

Random Variables

A random variable is a numeric quantity whose value depends on the outcome of a random event

- We use a capital letter, like X , to denote a random variable
- The values of a random variable are denoted with a lowercase letter, in this case x
- For example, $P(X = x)$

There are two types of random variables:

- **Discrete random variables** often take only integer values
Example: Number of credit hours, Difference in number of credit hours this term vs last
- **Continuous random variables** take real (decimal) values
Example: Cost of books this term, Difference in cost of books this term vs last

Lottery

```
library(DATA606)
shiny_demo('lottery')
```

Lottery Tickets

This application will simulate buying a series of lottery tickets. For example, the default starting point of 365 is meant to simulate buying one lottery ticket a day for a year. The "Odds" tab provides the exact odds of winning each ticket. Clicking the "New Run" button will simulate another "year" of buying tickets showing wins along the way and the total winnings or losses at the end. Past runs will be saved and plotted in light grey to show how the current run compares to previous runs.

Number of tickets:

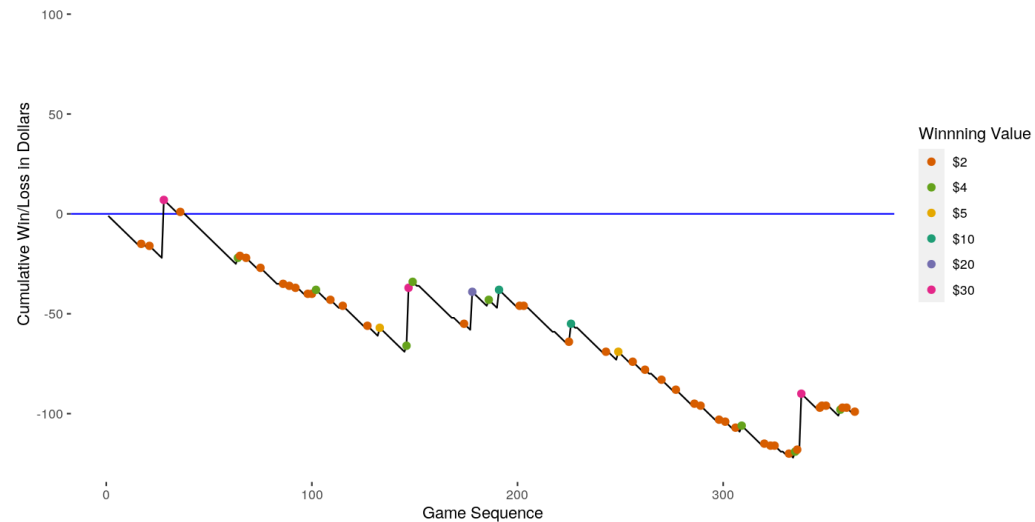


New Run

10 Runs

Plot Histogram Odds

Average losses after 365 games is \$100



Expectation

- We are often interested in the average outcome of a random variable.
- We call this the expected value (mean), and it is a weighted average of the possible outcomes

$$\mu = E(X) = \sum_{i=1}^k x_i P(X = x_i)$$

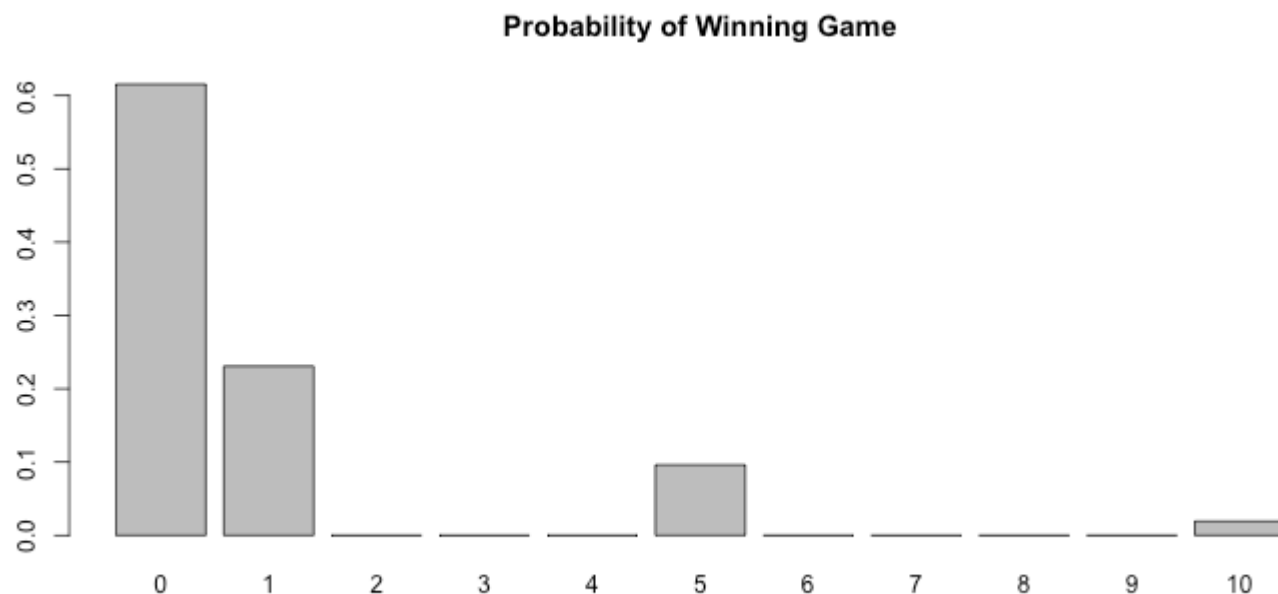
Expected value of a discrete random variable

In a game of cards you win \$1 if you draw a heart, \$5 if you draw an ace (including the ace of hearts), \$10 if you draw the king of spades and nothing for any other card you draw. Write the probability model for your winnings, and calculate your expected winning.

Event	X	P(X)	X P(X)
Heart (not Ace)	1	12/52	12/52
Ace	5	4/52	20/52
King of Spades	10	1/52	10/52
All else	0	35/52	0
Total	$E(X) = \frac{42}{52} \approx 0.81$		

Expected value of a discrete random variable

```
cards <- data.frame(Event = c('Heart (not ace)', 'Ace', 'King of Spades', 'All else'),  
  X = c(1, 5, 10, 0),    pX = c(12/52, 5/52, 1/52, 32/52) )  
cards$XpX <- cards$X * cards$pX  
cards2 <- rep(0, 11)  
cards2[cards$X + 1] <- cards$pX  
names(cards2) <- 0:10  
barplot(cards2, main='Probability of Winning Game')
```



Estimating Expected Values with Simulations

```
tickets <- as.data.frame(rbind(
  c(' $1', 1, 15),
  c(' $2', 2, 11),
  c(' $4', 4, 62),
  c(' $5', 5, 100),
  c(' $10', 10, 143),
  c(' $20', 20, 250),
  c(' $30', 30, 562),
  c(' $50', 50, 3482),
  c(' $100', 100, 6681),
  c(' $500', 500, 49440),
  c(' $1500', 1500, 375214),
  c(' $2500', 2500, 618000)
), stringsAsFactors=FALSE)
names(tickets) <- c('Winnings', 'Value', 'Odds')
tickets$Value <- as.integer(tickets$Value)
tickets$Odds <- as.integer(tickets$Odds)
```

Estimating Expected Values with Simulations

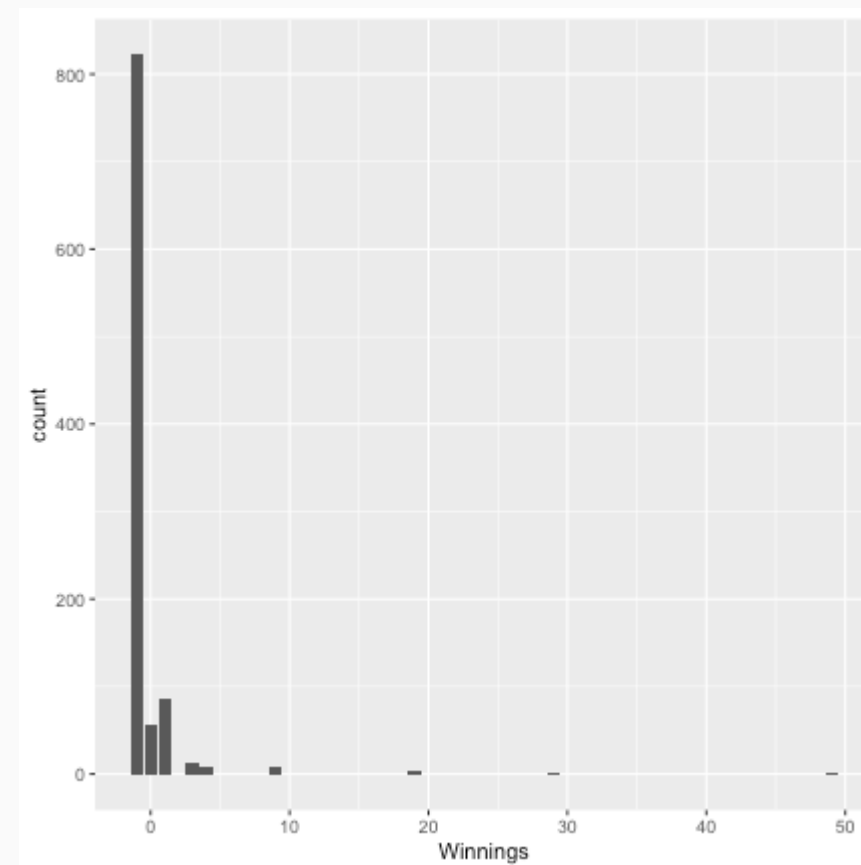
```
m <- 618000 * 375214 # A multiple of all odds
odds <- sample(m, 1000, replace=TRUE)
vals <- rep(-1, length(odds))
for(i in 1:nrow(tickets)) {
  vals[odds %% tickets[i,'Odds'] == 0] <-
    tickets[i,'Value'] - 1
}
head(vals, n=10)
```

```
## [1] -1  0 -1 -1 -1 -1 -1 -1 -1  0
```

```
mean(vals)
```

```
## [1] -0.436
```

```
ggplot(data.frame(Winnings=vals), aes(x=Winnings)) +
  geom_bar(binwidth=1)
```



Expected Value of Lottery Example

$$\mu = E(X) = \sum_{i=1}^k x_i P(X = x_i)$$

tickets

##	Winnings	Value	Odds	xPx
## 1	\$1	1	15	0.066666667
## 2	\$2	2	11	0.181818182
## 3	\$4	4	62	0.064516129
## 4	\$5	5	100	0.050000000
## 5	\$10	10	143	0.069930070
## 6	\$20	20	250	0.080000000
## 7	\$30	30	562	0.053380783
## 8	\$50	50	3482	0.014359563
## 9	\$100	100	6681	0.014967819
## 10	\$500	500	49440	0.010113269
## 11	\$1500	1500	375214	0.003997719
## 12	\$2500	2500	618000	0.004045307

Expected value for one ticket

```
sum(tickets$xPx) - 1
```

```
## [1] -0.3862045
```

Expected Value of Lottery Example (cont)

```
sum(tickets$xPx) - 1 # Expected value for one ticket
```

```
## [1] -0.3862045
```

Simulated

```
nGames <- 1
runs <- numeric(10000)
for(j in seq_along(runs)) {
  odds <- sample(max(tickets$Odds), nGames, replace = TRUE)
  vals <- rep(-1, length(odds))
  for(i in 1:nrow(tickets)) {
    vals[odds %% tickets[i,'Odds'] == 0] <- tickets[i,'Value'] - 1
  }
  runs[j] <- cumsum(vals)[nGames]
}
mean(runs)
```

```
## [1] -0.3781
```

Note on Randomization in R

We will use many different functions throughout the course to randomly generate data. The first is the `sample` function. This function simply randomly samples from the first parameter. Consider the `letters` vector containing the 26 letters of the alphabet. Calling `sample` with just that vector will shuffle the vector.

```
letters
```

```
## [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"  
## [20] "t" "u" "v" "w" "x" "y" "z"
```

```
sample(letters)
```

```
## [1] "u" "i" "p" "v" "l" "f" "h" "t" "b" "g" "s" "c" "j" "x" "a" "w" "m" "d" "o"  
## [20] "z" "q" "k" "r" "y" "e" "n"
```

Note on Randomization in R (cont.)

You can specify how many you want to return with the `size` parameter.

```
sample(letters, size = 1)
```

```
## [1] "x"
```

The `replace` will ensure that each randomly selected value is independent of the others.

```
sample(letters, size = 30, replace = TRUE)
```

```
## [1] "j" "k" "a" "n" "l" "f" "g" "l" "c" "t" "i" "c" "v" "e" "z" "k" "d" "r" "d"  
## [20] "g" "k" "t" "f" "j" "g" "j" "m" "h" "r" "z"
```

Coins Example

```
coin <- c('H', 'T')  
sample(coin)
```

```
## [1] "H" "T"
```

```
sample(coin, 1)
```

```
## [1] "T"
```

```
sample(coin, 100, replace = TRUE)
```

```
## [1] "T" "H" "H" "H" "T" "H" "T" "H" "H" "T" "H" "T" "H" "H" "H" "T" "T" "H"  
## [19] "H" "H" "T" "H" "H" "T" "T" "T" "H" "T" "T" "H" "T" "H" "T" "T" "T" "H"  
## [37] "H" "H" "H" "T" "H" "T" "H" "H" "H" "T" "T" "T" "H" "T" "H" "H" "T" "H"  
## [55] "T" "T" "H" "H" "H" "H" "H" "H" "H" "T" "H" "H" "T" "T" "H" "T" "T" "H"  
## [73] "H" "T" "H" "T" "H" "T" "H" "H" "H" "H" "H" "H" "H" "H" "T" "H" "H" "H"  
## [91] "T" "T" "H" "T" "H" "H" "H" "T" "T" "H"
```

Seeds

Computers are generally not good at randomization. Instead, R (and really all programs) uses a **pseudo random algorithm**. These algorithms rely on a seed, or starting point for the algorithm. You can set the seed to ensure that your analysis is reproducible. For example, setting the seed below before calling `sample` will ensure we get the same answer.

```
set.seed(2112); sample(100, 1)
```

```
## [1] 6
```

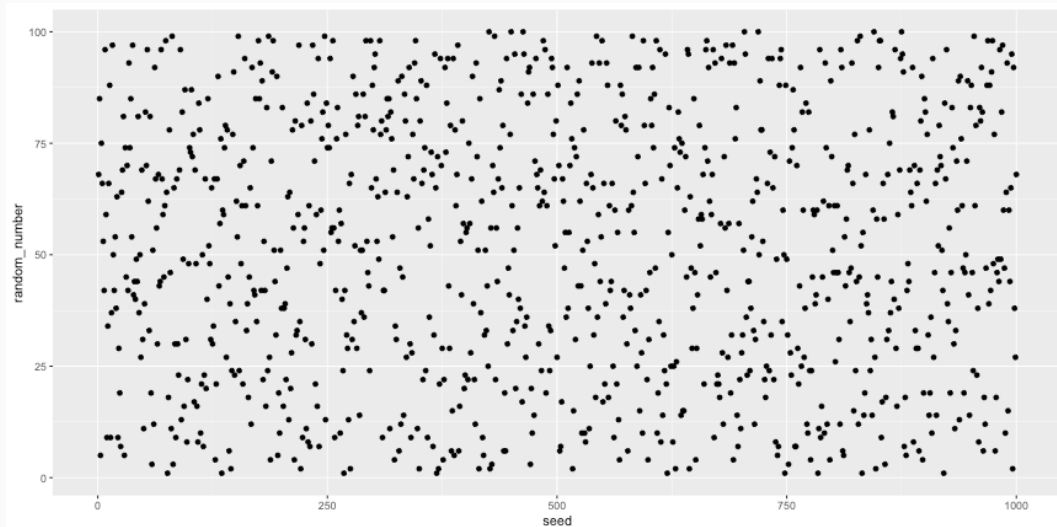
```
set.seed(2112); sample(100, 1)
```

```
## [1] 6
```

Is it really random?

```
df <- data.frame(seed = 1:1000, random_number = NA_integer_)
for(i in 1:nrow(df)) {
  set.seed(df[i,]$seed)
  df[i,]$random_number <- sample(100, 1)
}
```

```
ggplot(df, aes(x = seed, y = random_number)) + geom_point()
```



```
cor.test(df$seed, df$random_number)
```

```
##
##      Pearson's product-moment correlation
##
## data:  df$seed and df$random_number
## t = -0.11161, df = 998, p-value = 0.9112
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.06551171  0.05847292
## sample estimates:
##              cor
## -0.003532972
```

One Minute Paper

1. What was the most important thing you learned during this class?
2. What important question remains unanswered for you?



Scan the QR code or click here to
complete the One Minute Paper